

ECEN 428/722 - Lab Report

Lab Number: 5

Lab Title: JPEG Decoder Design on FPGAs

Section Number: 602

Student's Name: Sai Vikhyat Pattar

Student's UIN: 936001854

Date: 10/2/2025

TA: Cristhian Roman Vicharra

Objectives:

- Design the JPEG decoder
- For designing the JPEG decoder, first understand the zig zag encoding and huffman encoding
- Use this understanding of encoding to design inverse zig zag decoding and huffman decoding

Questions:

1) What is the purpose of using zigzag transformation in JPEG encoding?

A) The zigzag transformation rearranges the 2D block of DCT coefficients (after quantization) into a **1D sequence** by scanning diagonally from the top-left corner, where most of the low-frequency components are concentrated. This ordering groups similar frequency components together—placing **significant low-frequency values at the beginning** and **high-frequency (often zero) values at the end** of the vector. By clustering zeros toward the tail, this step facilitates **efficient run-length and entropy encoding**, improving overall compression efficiency.

2) What is the purpose of using Huffman encoding in JPEG encoding?

A) Huffman encoding, performed after the zigzag scan, is a **lossless compression technique** that assigns **variable-length binary codes** to symbols based on their frequency—shorter codes for frequent symbols and longer ones for rare symbols. This reduces the average number of bits required for representation, lowering the file size without any data loss. Since the codes are prefix-free, no delimiters are needed, and precomputed lookup tables (LUTs) on FPGA hardware allow for **fast encoding and decoding**. The result is **efficient, lossless data compression** with reduced storage and transmission requirements.

Results:

With a clear understanding of zigzag encoding and Huffman encoding, I now have the necessary foundation to design their corresponding inverse operations—**zigzag decoding** and **Huffman decoding**—which will enable the complete implementation of a **JPEG decoder**.